



Abstract

Projekttitle: Network Architect

Kurs: Software Engineering (DLMCSPSE01_D)

Student: Johannes Öllerer

Matrikelnummer: 92211477

Datum: 01.August 2026



Inhaltsverzeichnis

1 Einleitung und Projektkontext	3
2 Entwicklungsverlauf des Projekts	3
3 Herausforderungen und Problemlösung	4
4 Reflexion des Zeitplans	4
5 Lessons Learned und Ausblick.....	4

1 Einleitung und Projektkontext

Network Architect ist eine unter Windows 10/11 lauffähige Desktop-Anwendung, die das klassische Hashiwokakero-Puzzleprinzip mit einer Netzwerk-Thematik verbindet. Statt abstrakten Inseln und Brücken arbeiten die Spieler mit Netzwerkknoten wie Servern und Routern sowie verschiedenen Verbindungstypen, die Bandbreite und Redundanz abbilden. Ziel des Projekts war es, ein Puzzle-Spiel zu entwickeln, das Puzzle-Enthusiasten und IT-Studierenden eine anspruchsvolle, aber zugängliche Möglichkeit bietet, logisches Denken mit grundlegenden Netzwerk-Konzepten zu verknüpfen.

Das Projekt entstand im Portfoliokurs „Projekt: Software Engineering (DLMCSPSE01_D)“ und folgte den Phasen Konzeptions-, Erarbeitungs- und Finalisierungsphase. Dabei wurden Anforderungserhebung, Spezifikation, Architekturentwurf, Implementierung und Qualitätssicherung durch entsprechende Dokumente begleitet. Die Entwicklung orientierte sich an einem iterativen Wasserfallmodell, in dem Feedback aus Tutorien und Tests zu Anpassungen in Anforderungen, Spezifikation und Code führte und nicht vollständig erfüllte Anforderungen als technische Schuld festgehalten wurden.

2 Entwicklungsverlauf des Projekts

In Phase 1 wurden Projektdokumentation, Anforderungs- und Spezifikationsdokument erstellt und die grundlegende Struktur des Projekts festgelegt. Nach dem Tutor-Feedback zeigte sich, dass die zeitliche Planung nicht ausreichend detailliert war: Das Fehlen eines Gantt-Diagramms und einer expliziten Dokumentation des Zeitbedarfs pro Arbeitspaket machte eine nachträgliche Ergänzung notwendig, die zwar aufwendig war, aber zu einem präziseren Projektstrukturplan führte.

Phase 2 bildete den zeitlich anspruchsvollsten Abschnitt. Während Architektur-Design und MVVM-Struktur planmäßig abgeschlossen wurden, erwies sich die Umsetzung der Game-Logic, der Validierungsregeln und der QGraphicsView-basierten GUI als deutlich umfangreicher als angenommen. Die erstmalige Arbeit mit MVVM und das Erlernen von PySide6/QGraphicsView führten zu zusätzlichem Einarbeitungsaufwand, bildeten jedoch die Grundlage für eine stabile und klar strukturierte Anwendung.

In Phase 3 lag der Schwerpunkt auf Testdesign, Qualitätssicherung und der finalen Ausarbeitung der Dokumentation. Die konsequente Nutzung von pytest und der Testpyramide bereits während der Implementierung stellte sicher, dass viele Unit- und Integrationstests zu Beginn der Finalisierungsphase bereits vorhanden waren. Dadurch konnten Testdokument, Benutzeranleitung und Projektdokumentation mit geringem Nacharbeitsaufwand fertiggestellt und das Projekt im vorgesehenen Zeitrahmen abgeschlossen werden.

3 Herausforderungen und Problemlösung

Technisch standen vor allem zwei getrennte Algorithmen im Mittelpunkt: die Lösungsprüfung und die Berechnung der Bewertungsscores. Die grundlegende Prüfung, ob alle Nodes mit dem Server verbunden sind und dieser mindestens zwei Anbindungen besitzt, ließ sich vergleichsweise einfach umsetzen. Anspruchsvoller war hingegen die Entwicklung der beiden Bewertungsalgorithmen für PerformanceScore und RedundancyScore, die Bandbreite und Ausfallsicherheit eines Netzwerks getrennt erfassen. Hier waren mehrere Iterationen nötig, bis beide Verfahren stabil funktionierten und die gewünschten Unterschiede zwischen einfachen und qualitativ hochwertigen Netzwerkdesigns sichtbar machten. Zusätzlich stellte die Performance der Lösungsprüfung bei großen Puzzle-Layouts eine Herausforderung dar. Während typische Levelgrößen in akzeptabler Zeit validiert und bewertet werden konnten, führte die Prüfung sehr großer Netzwerke zu spürbar längeren Laufzeiten. Aufgrund des langen Projektzeitraums mit begrenzter verfügbarer Zeit wurde eine grundlegende algorithmische Optimierung verworfen; die funktionale Korrektheit wurde priorisiert, während die Performance bei sehr großen Levels als technische Schuld akzeptiert und dokumentiert wurde.

4 Reflexion des Zeitplans

Der Zeitplan wurde in Form eines Projektstrukturplans und eines Gantt-Diagramms mit klar definierten Arbeitspaketen ausgearbeitet. Im Verlauf zeigte sich, dass insbesondere die Implementierung von Game-Logic und GUI mehr Zeit benötigte als geplant, während andere Pakete schneller erledigt werden konnten. Trotz Abweichungen erwies sich die detaillierte Planung als wichtige Grundlage, um Prioritäten auf zentrale Aufgaben wie Spiellogik, Tests und Dokumentation zu legen und das Projekt geordnet zu steuern. Gleichzeitig zog sich das Projekt über einen langen Zeitraum, da die zeitliche Beanspruchung des Hauptberufs häufiger höher war als ursprünglich angenommen und geplante Arbeitsphasen für das Portfolio entsprechend verschoben werden mussten.

5 Lessons Learned und Ausblick

Aus dem Projekt ergeben sich zwei zentrale Erkenntnisse: Die MVVM-Architektur erwies sich als geeignetes Strukturmodell, um Spiellogik, Datenhaltung und GUI klar zu trennen und die Wartbarkeit zu sichern. Ebenso zeigte sich, dass die frühzeitige Einführung automatisierter Tests mit pytest und die Orientierung am AAA-Muster eine strukturierte und nachvollziehbare Qualitätssicherung ermöglichen. Das im Testdokument beschriebene System aus Unit-, Integrations- und UI-orientierten Tests erhöhte die Sicherheit bei Änderungen und machte die Testabdeckung transparent. Für zukünftige Projekte – oder eine weiterentwickelte Version von Network Architect – bildet diese Kombination eine Basis, auf der zusätzliche Features, Performance-Optimierungen und vertiefte Usability-Analysen schrittweise umgesetzt werden können.